Presentation and Viva Guide

Abdul Mueed (22SW058) / Abdul Aziz (22SW046)

Suggested speaking split: Abdul Mueed covers slides 1-4. Abdul Aziz covers slides 5-7 and the live demonstration. Both members should be able to explain the calculation.

Explain the project in this order

1. Scenario

You play the shop owner. Spend gold on ingredients and bottles before opening. Brew too little and customers leave empty-handed. Brew too much and you recover only part of the cost at closing. The simulation tests that batch decision over many shop days.

2. Inputs

Prices and probabilities adapt the supplied model. The fictional game rules are explicit; no real-game observations are claimed.

3. Calculation

Sold is the smaller of stock and demand. Cash includes crafting and recovery. The textbook score also deducts foregone margin on unmet demand.

4. Experiment

Thirty runs, one thousand shop days each. Eleven stock quantities share the same demands within each run.

5. Results

Textbook score chooses 70. Cash alone chooses 60. Show both and explain the distinction.

6. Live change

Change selling price from 50 to 80, run again and show the computed winner becomes 80.

7. Verification

Thirty-one passing tests, hand checks and an exact expectation provide evidence. The app shows comparison tables, per-run results and daily audits.

Likely questions

Can I test stock 500 or another range?
Yes. Set Minimum stock, Maximum stock and Stock step, then choose a quantity in that range. For 500, use 0-500 in steps of 10. The benchmark is the best within the chosen range. Workload guards protect the web request, not a fixed stock limit.

Why clear stock at closing?
Each simulated day is a separate one-day game challenge. A wholesaler pays 10 gold per leftover potion at closing. This is the terminal recovery rule, not a claim that potions naturally expire. A continuing campaign would need a carryover model.

Where do numbers come from?
The supplied course notes, probability table p.68 and rupee assignment p.75, relabelled as gold. The config and form expose the actual inputs.

Why two recommended quantities?
The textbook score penalizes unmet demand beyond cash accounting. Score maximization chooses 70; ordinary cash profit chooses 60.

Is the result hardcoded?
No. simulate.py evaluates every feasible batch, computes averages and selects all maximizers. The app calls the same engine. Price 80 changes the winner.

How many tests and simulations?
Thirty-one automated tests. The baseline contains thirty separately seeded runs of one thousand shop-day scenarios, evaluated for eleven quantities. Distinguish these counts.

How do you know it is correct?
Hand cases, reproduction of the source ten-day total, exhaustive digit-pair verification and finite exact expected-return calculations.

What does Django do?
Validate inputs, protect POST forms, call the Python engine, render the chart and on-page evidence tables. The browser does not calculate the simulation.

What are the key code functions?
simulate.py: pick maps rolls, generate_days samples demand, daily_result calculates one shop day, exact_metrics sums the finite expectation, experiment performs repeated comparisons. simulator/views.py connects these to the app.